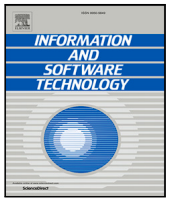
Information and Software Technology 162 (2023) 107263



Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof





Metamorphic testing of chess engines☆

Manuel Méndez *, Miguel Benito-Parejo, Alfredo Ibias, Manuel Núñez

Design and Testing of Reliable Systems Research Group, Complutense University of Madrid, Madrid, Spain

ARTICLE INFO

Keywords:
Software testing
Metamorphic testing
Chess engines

ABSTRACT

Context: Chess engines are computer programs that analyse chess positions. The goal of this analysis is to decide which player has an advantage and evaluate how big the advantage is. Using this analysis, chess engines are really powerful players who can consistently beat the best (human) players. Even though these programs are fantastic players, we cannot be sure that the code is fault free because it is very difficult to test them. In particular, we face the oracle problem: if the chess engine plays better than any potential tester, how can a tester claim that a certain evaluation is wrong or that a suggested move is not the best one?
Objective: The main goal of our work is to provide a metamorphic testing tool to evaluate chess engines. In particular, we are interested in looking for inconsistent behaviours in the best publicly available chess engine, Stockfish, but we would also like to consider other chess engines.
Methods: We developed a metamorphic testing solution to validate chess engines. First, we defined metamorphic relations that might reveal inconsistent behaviours. The underlying idea was that the evaluation of *related* positions should be the same. For example, if we consider a position and rotate all the pieces with respect to the central axis, then both positions should have the same evaluation. One of our main priorities was to have a fully automatised tool. *Source inputs* are obtained from available datasets while *follow-up inputs* are automatically computed by applying sound transformations to the source inputs with respect to the corresponding metamorphic rule. In order to assess the usefulness of our work, we applied it to analyse a dataset with more than 40,000 positions.
Results: Empirical evidence validates the usefulness of our work to analyse the best available chess engine, Stockfish. Our tool revealed non-negligible deviations from the expected behaviour in Stockfish for all the MRs. Additional experiments showed that our tool can be easily used to analyse other chess engines such as Komodo, Houdini and Gull.
Conclusion: The experiments demonstrate the usefulness of our approach to identify issues in the latest version of the widely recognised to be the best chess engine: Stockfish (version 15, released in April 2022). Our tool is flexible and can be easily extended with metamorphic relations that can be defined in the future by either us or other users. Since all our metamorphic relations are implemented and the code is freely available, users can use them as a pattern to implement new relations.

1. Introduction

Software Testing [1,2] is the most widely used validation technique to assess the quality of software systems. Usually, the tester supplies inputs to the System Under Test (SUT), observes the outputs produced after processing these inputs and evaluates whether these outputs are the ones that the SUT was supposed to produce. There are several problems underlying the apparently simple testing process that we have just described. For example, how many inputs do we need to supply? How do we prioritise inputs? How do we ensure that the SUT has been sufficiently covered? In this paper we consider how the decision on the correctness of the obtained outputs is made. Classically, testing was a mainly manual task where the tester selected the *best* inputs, supplied them to the SUT, and decided whether or not the observed outputs were expected. This process is time consuming and cannot be applied to complex systems. Sometimes the tester can rely on an *oracle* (i.e. a (partial) specification of the desired system) that provides a verdict on the observed behaviour. Moreover, if we have a specification then we can improve the automation of the testing process because,

☆ This work has been supported by the State Research Agency (AEI) of the Spanish Ministry of Science and Innovation under grant PID2021-122215NB-C31 (AwESOMe); the Region of Madrid under grant S2018/TCS-4314 (FORTE-CM) co-funded by EIE Funds of the European Union; and Santander–Complutense University of Madrid under grant CT82/20-CT83/20.

* Corresponding author.
E-mail addresses: manumend@ucm.es (M. Méndez), mibeni01@ucm.es (M. Benito-Parejo), aibias@ucm.es (A. Ibias), manuelnu@ucm.es (M. Núñez).

https://doi.org/10.1016/j.infsof.2023.107263
Received 31 October 2022; Received in revised form 13 May 2023; Accepted 22 May 2023
Available online 5 June 2023
0950-5849/© 2023 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

in particular, unexpected outputs are automatically detected. Actually, *Model-Based Testing* is a very active field of research [3–5]. Usually, testers do not have oracles [6] and the testing process relies on them to decide whether the observed outputs are expected. Unfortunately, there are situations where even an expert tester would be unable to decide if the correct outputs are observed. This is the type of systems that we study in this paper.

During the last 25 years [7], Metamorphic testing [8–10] (MT) has proven useful to alleviate the oracle problem. Instead of sequentially supplying inputs and deciding whether the expected outputs are observed, MT collectively analyses a set of inputs and the obtained outputs to check whether certain relations between supplied inputs and observed outputs, called *Metamorphic Relations* (MRs), are violated. The classical example to motivate the use of MT is the sine function where we might not know the exact value of $\sin(\alpha)$, for a certain $\alpha \in \mathbb{R}$, but where we can establish relations between different inputs such as $\sin(\alpha) = \sin(2 \cdot \pi + \alpha)$ or $\sin(-\alpha) = -\sin(\alpha)$. As we will see in the next section, MT has shown to be a very versatile technique to test complex systems in many heterogeneous and unrelated fields [11,12].

In this paper we use MT to evaluate chess engines. A chess engine is a software program that analyses a position in the board and proposes the best following moves. Chess engines usually evaluate the position giving a value in centipawns (cp). A positive valuation implies that White has an advantage while a negative one implies a Black advantage. For example, if a position is evaluated as $\{cp : +362\}$, then White has an advantage equivalent to 3.62 pawns. In other cases, the position is not evaluated according to cp, but according to $mate$. This means that checkmate can be forced in several moves. Again, a positive number means a White advantage and a negative number denotes a Black advantage. For example, $\{mate : -5\}$ means that Black can force checkmate in five moves. Since any program that evaluates a position is an engine, the proposed moves may vary among different engines. Determining which engine provides the best analysis is often studied by playing games between engines. Since 2010 the main reference for chess engines has been the *Top Chess Engine Championship* (TCEC) [13]. Although many engines participate, the most stable and consistent winner is Stockfish [14].[1] Therefore, Stockfish is the main open source, freely available, chess engine reference for human players. It is important to note that in 2018 AlphaZero beat Stockfish in 155 games out of 1000, lost in 6 games and there was a draw in the remaining 839 games. Unfortunately, AlphaZero belongs to DeepMind and it is currently not possible to either perform experiments or test this chess engine.

How do we test a chess engine? A usual technique is to use SPRT [15,16] (Sequential Probability Ratio Test). Essentially, if we want to test a chess engine we play a large number of games between the *engine under test* and a *reference* chess engine. Usually, the goal of this approach is to test the improvements of a newer version of a chess engine with respect to the previous one [17] using the ELO score — a method for calculating the relative skill of chess players [18]. However, when the reference chess engine plays much better than the vast majority of humans (who could be involved in testing) and we do not have anything to compare with, how do we test the last version of such chess engine? Two of the authors of this paper are frequent users of Stockfish and, despite being far superior to them, they have found unusual behaviours. For example, Fig. 1 (top) displays a position in which White only needs to continuously move the king to achieve a draw while Stockfish considers that there is a decisive advantage (1400 centipawns) for Black. Another less obvious example is shown in Fig. 1 (bottom). In this case, a precise play from Black ensures a draw. However, this position is evaluated by Stockfish as a slight advantage (50 centipawns) for White. Finally, a well-known specific sequence of

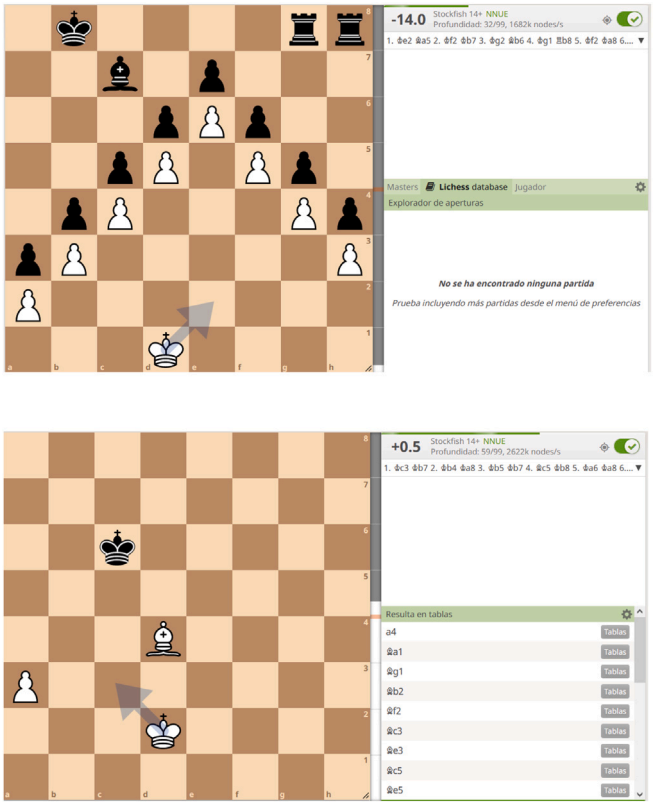


Fig. 1. Draw positions incorrectly evaluated by Stockfish (via lichess).

plays by White, including the sacrifice of knights and bishops, leads to a win against Stockfish in 17 moves.[2] This is caused by a poor choice of moves by Stockfish in a specific game situation.

Clearly, there is faulty behaviour by the chess engine. However, instead of using the term *error*, what might be misunderstood, we think that it is more appropriate to use an expression such as *deviation from the expected result*, to show that the chess engine is working properly but that the result deviates from the *expected* one. We would like to methodologically test the last available version of Stockfish. Given that we are testing a complex software system for which there is no oracle available, we decided to use MT. Intuitively, our MRs state that if we have two *equivalent* positions then the valuation of the positions should be almost the same. The intended meaning of "almost" in the previous statement is that we do not consider that small deviations represent a faulty behaviour. Similarly, the SUT should suggest the same next best moves if we consider equivalent positions, more precisely, the evaluation of two equivalent positions after performing the best suggested move should be almost the same. We consider *thresholds* to discard small differences between the expected and the observed outputs.

In order to evaluate the usefulness of our work, we performed a large number of experiments. We used three well-known sets of positions, our *source inputs*, produced several equivalent positions for each position, our *follow-up inputs*, and evaluated whether the corresponding outputs were similar. Even with the use of very conservative thresholds, we were able to find non-negligible deviations in 1.8% of the experiments. If we set lower thresholds, the most effective MR was able to find deviations in up to 43.5% of the cases.

The rest of the paper is organised as follows. In Section 2 we review related work. In Section 3, we informally introduce the main Stockfish features. We also present formal definitions of Stockfish inputs and

[1] In particular, Stockfish has won the last 5 editions of the TCEC and 10 out of the 12 last editions of the TCEC (being runner-up in the remaining two editions).

[2] This game is found in https://www.youtube.com/watch?v=C5ul6b695Pw.

outputs and the general pattern of our MRs. In Section 4 we present our MRs and provide implementation details. In Section 5 we pose our research questions, discuss the results of our experiments, answer our research questions and analyse threats to the validity of the obtained results. Finally, in Section 6 we present our conclusions and sketch some lines for future work.

2. Related work

MT has been successfully applied in very different fields to test complex systems where oracles are not available. Recent MT contributions illustrate its versatility. MT has been used in citation database systems [19], cryptography functions [20], debuggers [21], image recognition systems [22], machine learning classifiers [23], machine translation services [24], maps [25], natural language processing [26], neural networks [27], quantum computing [28] and sentiment analysis systems [29].

Of special interest to us are MT approaches analysing systems that output *numbers* that are not trivial to compute. MT has been used to analyse Geographic Information Systems that compute superficial areas [30], numerical simulation programs [31], the density of material defects [32] and different performance measures [33,34]. These systems share a characteristic with our work: the use of *thresholds*.

Some of them work with floating point calculations and, therefore, slight variations in the output cannot be considered system errors. Some also need to take into account non-deterministic behaviour associated with non-functional properties, such as execution time, memory consumption and energy usage, that depend on the SUT's configuration. In our case, thresholds are used to disregard small differences between the expected and the observed outputs. For example, this is the case if the values corresponding to a source input and a follow-up input that is a valid rotation of the first one are *almost the same*. We should also mention recent work on testing OpenStreetMap where thresholds are used to specify how many elements a certain set has to be considered big or small [35].

It is important to emphasise that symmetries, a feature underlying some of our MRs, have been already exploited in MT [36]. This work points out that symmetry is a universal property and, based on it, proposes several symmetry-based MRs that can detect failures in some of the world most popular applications.

To the best of our knowledge, MT has been used only once in chess [37,38]. However, the goal and the approach is completely unrelated to ours. The MRs presented in that work focus on checking that, given a specific board state, all the moves allowed by the system are valid and that the system does not allow invalid moves. That is, the target of their MRs is the implementation of the game, in the sense that it is executed as the rules of chess dictate. In their first work [37], the authors focused on an implementation that allowed to play chess, that is, an interface that allows moves. They evaluated the capability of their MRs to detect erroneous moves by introducing faults in the code of the SUT applying mutation testing techniques [39]. They were able to detect injected faults when some illegal move was performed. In the subsequent paper [38], the authors considered an open source engine that uses an alpha–beta pruning algorithm. They performed mutation testing by mutating logical, mathematical, relational and shift operators. Their MRs were able to detect 71% of injected faults, as they either triggered illegal moves or made the alpha–beta pruning inconsistent. In our case, we do not focus on the validity of the moves because our goal is to evaluate the consistency of the evaluation of an engine, a more difficult problem. Our aim is to check whether two *similar* positions have *similar* scores, where the meaning of similar varies depending on the MR in each scenario and the way the follow-up input is produced. Since we do not inject faults, we do not need to consider the chess engines as white-boxes. In fact, we fully consider our SUTs to be black-boxes. Overall, their research differs from ours because we focus on the evaluation that the engine yields, which indicates who has the advantage (and how much of an advantage they have), whereas [37,38] only focus on the validity of the moves. It is worth to mention that our first experiments with Stockfish evaluated whether it allowed invalid moves starting from a valid position: we did not find any. However, as we will explain later, Stockfish considers to be a *valid* position a situation where both kings are checked. This is not allowed by chess rules and we should discard these positions.

There are some specific approaches to test chess engines. As we mentioned before, Sequential Probability Ratio Test methods [15,16] and Gauntlet tournament testing are usually used [40]. These methods focus on making the engine play multiple matches against either itself (or a previous version of itself) or against other already well established chess engines. The goal, thus, is to test whether the SUT is better than other chess engines at playing the game and, indirectly, at analysing it. As we explained in the introduction, these techniques cannot be used if we are testing the best available chess engine: just being better than the rest does not mean that it is free of faults.

3. Preliminaries

In this section we describe the main features of Stockfish. In addition, we present the basis of our MT approach. We introduce a formal notation to define the inputs and outputs of Stockfish and present the generic pattern that we use for our MRs. The formal definition of our MRs is outlined in Section 4.

3.1. Stockfish

Stockfish [14] is a state-of-the-art chess engine. It is worth to mention that the system is freely available and open source. Its first version was released in November 2008. Since then, fourteen additional versions have been released with Stockfish 15 available since April 2022 and ahead of Stockfish 14 by 36 ELO points. This is the version that we evaluate in this paper. In addition to be a consistent winner of the TCEC, we can highlight a match between Stockfish 8 and Hikaru Nakamura, one of the strongest chess players in history. In the first two games, Nakamura was helped by Rybka, a chess engine very popular in those years, and in the last two games he had positional advantage. Despite this, Stockfish won the match 3-1 (two wins of Stockfish and two draws).

Stockfish is a deterministic single-threaded chess engine and all the experiments that we performed use a single thread. Stockfish is so superior to other chess engines that it is used in the most famous chess websites, such as and , as a server-side program. This allows users of these websites to evaluate their own games and deeply analyse openings and endgame positions.

All the code needed to implement our tool is written in Python [41] and it is freely available. We used a package that provides an easy-to-use class to integrate Stockfish with Python. This package contains several simple functions that are invoked by our code. For example, we have functions to set a position by using Forsyth–Edwards Notation (FEN [42]), check whether a position is valid, perform a sequence of moves from a position, get a board visual from a position, find what piece is in a certain square, or find whether a move is a capture and what type of capture. We make intensive use of two more complex functions. The first one, `stockfish.get_evaluation()`, returns the evaluation from a given position while the second one, `stockfish.get_top_moves(n)`, returns the sorted n best moves from a given position.

In order to use the previously mentioned functions, we have to set some parameters used by Stockfish: *ELO rating*, providing Stockfish with a level similar to a player with that rating, *skill level*, equivalent to the ELO rating but using a different scale, and *depth*, that is, the number of half moves (a move made by one side) that the engine looks ahead to choose the best next move and evaluate the (dis)advantage of one player in the current position. Note that ELO rating is ignored if we set a skill level and viceversa. Generally speaking, a higher ELO rating/skill level and depth implies a higher accuracy by Stockfish.

3.2. Formal definitions

In order to present an MT approach, we need to formally define all the involved components. These include SUT inputs, SUT outputs, and a general pattern for MRs.

A board is represented as a single string in the standard Forsyth–Edwards Notation (FEN), with blanks separating the different elements of the tuple (with possible blanks as well if no castling nor *en passant* captures are available at such position [43]). For example, the board given in Fig. 1 (bottom) is represented by the string

'8/8/2k5/8/3B4/P7/3K4/8 w'

assuming that White moves. In our experiments, we will not consider positions where castling or *en passant* captures are available because the *symmetries* that are needed to apply our MRs might be lost. For example, we might consider a position where castling is allowed but such that it is not allowed in a rotation of this position. Note that this is not a shortcoming of our proposal because we are able to reveal faults without using this position type.

Definition 1. A triple (p, d, l) is an *input* if p is an FEN string with the board information, $d \in \mathbb{N}$ is the analysis depth, meaning the number of future moves the engine evaluates, and $l \in \mathbb{N}$ is the analysis ELO. We denote the set of all inputs by $\mathcal{I}$.

The next step is to define what an *evaluation* is. Intuitively, given a position, an evaluation tells us which player has an advantage. In particular, if there exists a sequence of moves such that one player can force checkmate, then it identifies this by providing the smallest number of needed moves.

Definition 2. An *evaluation* $e \in \mathbb{Z} \cup (\{w, b\} \times \mathbb{N})$ represents the advantage one player has over the other. If e is a positive number, then we say White has a lead over Black of e centipawns; if e is a negative number, then Black has a lead over White of $-e$ centipawns. If $e = 0$ then we say that the position is even. Finally, if $e = (w, x)$ (respectively $e = (b, x)$) we say that White (respectively Black) has checkmate in x moves. A *move* is a string that indicates a valid movement for a board following the rules of chess [43]. Given a position p and a move m we denote by p `AFTER` m the position resulting after performing the move m at position p.

We say that the *application* of an input $i = (p, d, l) \in \mathcal{I}$ to the Stockfish engine, denoted by $\texttt{StockFish}(i)$, produces an *output* (e, m), where e is the resulting evaluation and m is the best proposed next move. We denote by $\mathcal{O}$ the set of all outputs.

Let us note that Stockfish can return the 5 best next moves but in our tool we only consider the best suggested move.

Next, we present our notion of MR. As usual, our MRs relate inputs and outputs. In our case, they relate two inputs and their corresponding outputs. A distinguishing feature with respect to usual MT approaches is the use of thresholds. Our first experiments showed that Stockfish provided different evaluations, in centipawns, for many *equivalent* inputs. Therefore, we decided to be more conservative and allow deviations from the expected result. Even though Stockfish is deterministic when running single threaded, and we performed some experiments that did not disprove this property, we realised that it provided slightly different evaluations for positions that were essentially the same. We understand that these small differences do not come from an actual fault, but instead from the way the heuristic analyses different positions. The fact is that, although we consider and understand that those positions are essentially equivalent, they are not the exact same positions. Therefore, the heuristic underlying the considered chess engine can analyse them in an slightly different way (for example, exploring the branches of the tree of future movements in a different order). To account for this kind of situations, we decided to consider thresholds. Specifically, we consider two thresholds associated with two measures of the deviation. The first one allows small deviations between evaluations in relative terms while the second one, in case the first one fails, allows small deviations in absolute terms. Intuitively, there is not much difference between having an advantage of 10 pawns and an advantage of 11 because in both cases the player has a clear advantage and will win the game. In contrast, having an advantage of 0.5 pawns versus an advantage of 1.5 is a completely different situation: an advantage of 0.5 pawns indicates that the player has a slight advantage while 1.5 pawns is a definitive advantage for competent players. The best way to interpret these thresholds is twofold. On the one hand, the looser the threshold, the more critical the faults detected, as they generate a bigger difference between the analysis of positions that should be equivalent. On the other hand, the tighter the threshold, the less critical the faults detected, as they produce smaller differences in the analysis.

Definition 3. Let $x, y \in \mathbb{Z}$ and $d \in \mathbb{R}$. We define the following predicates:

- $dist_{rel}(x, y, d)$ holds iff $\frac{|x-y|}{|x|+|y|} \leq d$.
- $dist_{abs}(x, y, d)$ holds iff $|x - y| \leq d$.

In the previous definition we have that

$$\max_{\{x,y \in \mathbb{Z}\}} \left\{ \frac{|x-y|}{|x|+|y|} \right\} = 1$$

and this value is reached only if x and y have opposite signs. In our context this can happen, for example, if the chess engine returns an evaluation of n centipawns for a position and of $-m$ centipawns for an equivalent position, with $n, m \in \mathbb{N}$.

Definition 4. A *metamorphic relation* for chess engines $R \subseteq \mathcal{I}^2 \times \mathcal{O}^2 \times \mathbb{R}^2$ is a relation over two inputs, their corresponding outputs, and two thresholds. Usually, we write $R(i_1, i_2, o_1, o_2, \delta, \epsilon)$ to denote $(i_1, i_2, o_1, o_2, \delta, \epsilon) \in R$.

Let δ and ϵ be thresholds, $i_1 = (p_1, d_1, l_1)$ and $i_2 = (p_2, d_2, l_2)$ be inputs, and $o_1 = \texttt{StockFish}(i_1) = (e_1, m_1)$ and $o_2 = \texttt{StockFish}(i_2) = (e_2, m_2)$ be the corresponding outputs. The definition of an MR R follows this pattern:

$$\begin{array}{c} R(i_1, i_2, o_1, o_2, \delta, \epsilon) \\ \Vert_{\mathbf{def}} \\ C(\delta, \epsilon, p_1, d_1, l_1, p_2, d_2, l_2, e_1, e_2, m_1, m_2) \end{array}$$

where C is a boolean expression. In the previous expression, we say that i_1 is a *source input* while i_2 is a *follow-up input*.

There is an important distinction between source and follow-up inputs. Usually, source inputs are provided by testers while follow-up inputs are (semi-)automatically generated from source inputs. In our case, we take source inputs from different position databases while follow-up inputs are automatically generated by applying *sound*, with respect to the corresponding MR, transformations to the source input.

4. Metamorphic Relations for chess engines

In this section we present our MRs. We provide formal definitions of the MRs currently included in our tool and discuss different alternatives that we used in our preliminary experiments but that are not used in the experiments reported in this paper.

Our MRs strongly rely on a specific relation between a source input and the corresponding follow-up input. Our first MR, $\texttt{MR}_{\texttt{equi}}$, considers that both inputs are *equivalent*, that is, there is no reason to produce different outputs when evaluating these inputs. We use two transformations to produce equivalent inputs (symmetry with respect to the axis and with respect to the $a1 - h8$ diagonal). In our second MR, $\texttt{MR}_{\texttt{mirror}}$, we use only one transformation to produce follow-up inputs: produce the mirror image of the position. In our third MR, $\texttt{MR}_{\texttt{better}}$, we produce follow-up inputs by replacing either a rook or

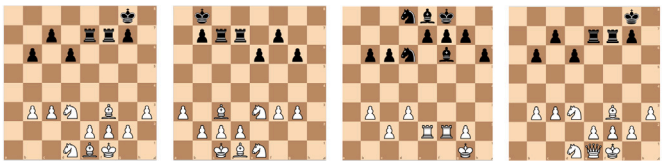


Fig. 2. Positions p, sim_axis(p), sim_mirror(p) and REPLACE(p). If White moves in p then Black moves in sim_mirror(p) and viceversa.

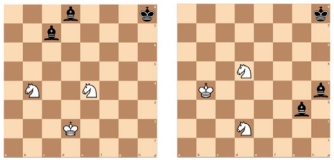


Fig. 3. Positions p and sim_diag(p).

a bishop by a queen. Finally, in our fourth MR, $\texttt{MR}_{\texttt{first}}$, we consider again a transformation of the original position such that we compare two positions after the best suggested move is performed.

In general, we can consider two types of MRs that can be used to evaluate a certain SUT. First, we have MRs based on *symmetries*. The goal is to exploit that similar inputs should produce similar results. These MRs are generic, the idea has been previously used in other domains, and it should be possible to use them, with small adjustments, in other board games. Our first two MRs, $\texttt{MR}_{\texttt{equi}}$ and $\texttt{MR}_{\texttt{mirror}}$ belong to this group. The second group of MRs are specific for chess engines. In this case, it is necessary to have a deeper knowledge of the chess domain to define sound MRs. Our last two MRs, $\texttt{MR}_{\texttt{better}}$ and $\texttt{MR}_{\texttt{first}}$, belong to this group. It is interesting to point out that some of the MRs belonging to this group might not be used at all in other board games (e.g. $\texttt{MR}_{\texttt{better}}$ makes no sense in Go) or might need strong adjustments (e.g. a version of $\texttt{MR}_{\texttt{better}}$ for shogi needs to take into account the value of the different figures), while other MRs might be easily adapted to other board games, as it is the case of $\texttt{MR}_{\texttt{first}}$.

Next, we formally define the transformations that are used in our MRs.

Definition 5. Let p be a position. We introduce the following notation:

- sim_axis(p) denotes the position where we rotate all the pieces with respect to the central axis. Fig. 2 displays an example.
- sim_diag(p) denotes the position where we rotate all the pieces with respect to the $a1 - h8$ diagonal. Note that we use this transformation only if there are no pawns in the board. Fig. 3 displays an example.
- sim_mirror(p) denotes the position where we exchange black and white pieces (and rotate the board). Fig. 2 displays an example.
- REPLACE(p) denotes the set of positions such that either one rook or a bishop of a given colour appearing in p is replaced by a queen of the same colour. Abusing the notation, we denote by replace(p) each of the members of REPLACE(p). Note that we use this transformation only if the replacement does not induce a stalemate. Fig. 2 displays an example where we replace the bishop at $e1$ by a white queen. In addition, we write $W(\texttt{replace}(p))$ (resp. $B(\texttt{replace}(p))$) if the colour of the replaced piece is white (resp. black).

4.1. Definition of our MRs

In this section we present formal definitions for our four MRs. Note that in all the MRs, in order to compare *similar* evaluations, both inputs have the same depth and ELO rating.

4.1.1. $\texttt{MR}_{\texttt{equi}}$: Assessing equivalent positions

This MR aims to detect situations where two *equivalent* positions have different evaluations. Two symmetric positions are equivalent if any move one player can perform on one board can be executed following the applied symmetry on the second one. It is important to note that a vertical and horizontal symmetry (hence diagonal) holds over the moves that all the pieces have except the pawns, which only holds horizontally. As such, the sim_diag can be applied only when there are no pawns on the board. For example, the knight move $\♘d1 - b2$ from the first board of Fig. 2 can be performed as $\♘e1 - g2$ in the second board, which is a transformation of the first board using sim_axis. Similarly, the move $\♗d8 - e7$ in the first board of Fig. 3 can be performed in the second board, which is a transformation of the first board using sim_diag (i.e. $\♗h4 - g5$).

Let $i_1 = (p_1, d, l) \in \mathcal{I}$ be a source input and $i_2 = (p_2, d, l) \in \mathcal{I}$ be a follow-up input such that either $p_2 = \texttt{sim_axis}(p_1)$ or $p_2 = \texttt{sim_diag}(p_1)$ and there are no pawns in p_1. Let $o_1 = \texttt{StockFish}(i_1) = (e_1, m_1)$ and $o_2 = \texttt{StockFish}(i_2) = (e_2, m_2)$. The definition of $\texttt{MR}_{\texttt{equi}}$ is found in Fig. 4.

The inputs for this MR are generated by choosing one position and fixing a depth and a level of analysis. Then, for the follow-up input, we keep the depth and level, but change the position by applying either sim_diag, only if there are no pawns, or sim_axis.

4.1.2. $\texttt{MR}_{\texttt{mirror}}$: Assessing positions where we exchange Black and White

This MR aims to detect situations where two mirrored positions return different evaluations. A mirrored position is one that is symmetric to another vertically, but with different colours and with the opposite player to move. Note that a position with Black/White to move is mirrored to a position with White/Black to move. For example, the knight move $\♘d1 - b2$ from the first board of Fig. 2 can be performed as $\♘d8 - b7$ in the third board, where the move corresponds to the opposite player.

Let $i_1 = (p_1, d, l) \in \mathcal{I}$ be a source input and $i_2 = (p_2, d, l) \in \mathcal{I}$ be a follow-up input where we have $p_2 = \texttt{sim_mirror}()$. Let $o_1 = \texttt{StockFish}(i_1) = (e_1, m_1)$ and $o_2 = \texttt{StockFish}(i_2) = (e_2, m_2)$. The definition of $\texttt{MR}_{\texttt{mirror}}$ is displayed in Fig. 4.

The inputs for this MR are generated in a similar manner to what is done in the previous MR by taking into account that the follow-up input is produced by applying the sim_mirror transformation.

4.1.3. $\texttt{MR}_{\texttt{better}}$: Assessing positions where we replace a piece by a better one

This MR aims to detect whether having a better position yields at least the same evaluation. We consider that a position where a rook or a bishop has been replaced by a queen can only improve the evaluation, as the queen can perform more moves than the original piece. For example, the fourth board in Fig. 2 has a queen in $e1$ that can make the same moves as the bishop in $e1$ in the first board, but the queen could also move through the e file if the $e2$ pawn disappeared.

Let $i_1 = (p_1, d, l) \in \mathcal{I}$ be a source input and $i_2 = (p_2, d, l) \in \mathcal{I}$ be a follow-up input where we have $p_2 \in \texttt{REPLACE}(p_1)$ and p_2 does not represent a stalemate. Let $o_1 = \texttt{StockFish}(i_1) = (e_1, m_1)$ and $o_2 = \texttt{StockFish}(i_2) = (e_2, m_2)$. The definition of $\texttt{MR}_{\texttt{better}}$ is displayed in Fig. 4. Recall that $W(p_2)$ (resp. $B(p_2)$) denotes that the replaced piece was white (resp. black).

The generation of follow-up inputs is similar, again, to the previous ones but taking into account that the chosen input belongs to $\texttt{REPLACE}(p_1)$.

4.1.4. $\texttt{MR}_{\texttt{first}}$: Assessing equivalent positions after the best suggested move

This MR aims to detect whether the evaluation of two related positions is preserved after a move. The evaluation of a position provides the information of the advantage of a player at a given position. After performing *the best move*, as suggested by Stockfish, such advantage should be mostly preserved. We have considered only the sim_axis transformation but it is immediate to extend the MR to also consider the sim_diag and sim_mirror transformations.

$$\texttt{MR}_{\texttt{equi}}(i_1, i_2, o_1, o_2, \delta, \epsilon)) =_{\textbf{def}} \begin{cases} \texttt{False} & \text{if } e_1 \in \{w,b\} \times \mathbb{N} \wedge e_2 \in \mathbb{Z}) \\ \texttt{False} & \text{if } e_2 \in \{w,b\} \times \mathbb{N} \wedge e_1 \in \mathbb{Z}) \\ e_1 = e_2 & \text{if } e_1, e_2 \in \{w,b\} \times \mathbb{N} \\ dist_{rel}(e_1, e_2, \delta) \vee dist_{abs}(e_1, e_2, \epsilon) & \text{if } e_1, e_2 \in \mathbb{Z} \end{cases}$$

$$\texttt{MR}_{\texttt{mirror}}(i_1, i_2, o_1, o_2, \delta, \epsilon) =_{\textbf{def}} \begin{cases} \texttt{False} & \text{if } e_1 \in \{w,b\} \times \mathbb{N} \wedge e_2 \in \mathbb{Z} \\ \texttt{False} & \text{if } e_2 \in \{w,b\} \times \mathbb{N} \wedge e_1 \in \mathbb{Z} \\ x_1 \neq x_2 \wedge y_1 = y_2 & \text{if } e_1 = (x_1, y_1), e_2 = (x_2, y_2) \in \{w,b\} \times \mathbb{N} \\ dist_{rel}(e_1, -e_2, \delta) \vee dist_{abs}(e_1, -e_2, \epsilon) & \text{if } e_1, e_2 \in \mathbb{Z} \end{cases}$$

$$\texttt{MR}_{\texttt{better}}(i_1, i_2, o_1, o_2, \delta, \epsilon) =_{\textbf{def}} \begin{cases} \texttt{False} & \text{if } e_1 \in \{w\} \times \mathbb{N} \wedge e_2 \in \mathbb{Z} \wedge W(p_2) \\ \texttt{True} & \text{if } e_1 \in \{w\} \times \mathbb{N} \wedge e_2 \in \mathbb{Z} \wedge B(p_2) \\ \texttt{False} & \text{if } e_1 \in \{b\} \times \mathbb{N} \wedge e_2 \in \mathbb{Z} \wedge B(p_2) \\ \texttt{True} & \text{if } e_1 \in \{b\} \times \mathbb{N} \wedge e_2 \in \mathbb{Z} \wedge W(p_2) \\ y_1 \geq y_2 & \text{if } e_1 = (x_1, y_1), e_2 = (x_2, y_2) \in \{w,b\} \times \mathbb{N} \\ & \quad \wedge x_1 = x_2 \wedge x_1(p_2) \\ y_1 \leq y_2 & \text{if } e_1 = (x_1, y_1), e_2 = (x_2, y_2) \in \{w,b\} \times \mathbb{N} \\ & \quad \wedge x_1 = x_2 \wedge \neg x_1(p_2) \\ dist_{rel}(e_1, e_2, \delta) \vee dist_{abs}(e_1, e_2, \epsilon) & \text{if } e_1, e_2 \in \mathbb{Z} \end{cases}$$

$$\texttt{MR}_{\texttt{first}}(i_1, i_2, o_1, o_2, \delta, \epsilon) =_{\textbf{def}} \begin{cases} \texttt{False} & \text{if } e'_1 \in \{w,b\} \times \mathbb{N} \wedge e'_2 \in \mathbb{Z} \\ \texttt{False} & \text{if } e'_2 \in \{w,b\} \times \mathbb{N} \wedge e'_1 \in \mathbb{Z} \\ x_1 = x_2 \wedge y_1 = y_2 & \text{if } e_1 = (x_1, y_1), e_2 = (x_2, y_2) \in \{w,b\} \times \mathbb{N} \\ dist_{rel}(e'_1, e'_2, \delta) \vee dist_{abs}(e'_1, e'_2, \epsilon) & \text{if } e_1, e_2 \in \mathbb{Z} \end{cases}$$

where $(e'_1, m'_1) = \texttt{StockFish}(p_1 \texttt{ AFTER } m_1, d, l)$ and $(e'_2, m'_2) = \texttt{StockFish}(p_2 \texttt{ AFTER } m_2, d, l)$.

Fig. 4. Definition of the MRs.

Let $i_1 = (p_1, d, l) \in \mathcal{I}$ be a source input and $i_2 = (p_2, d, l) \in \mathcal{I}$ be a follow-up input where we have $p_2 = \texttt{sim_axis}(p_1)$. Let $o_1 = \texttt{StockFish}(i_1) = (e_1, m_1)$ and $o_2 = \texttt{StockFish}(i_2) = (e_2, m_2)$. The definition of $\texttt{MR}_{\texttt{first}}$ is found in Fig. 4.

As expected, the follow-up inputs for this MR are computed in the same manner as $\texttt{MR}_{\texttt{equi}}$. An important difference with respect to the previous MRs is that the move that Stockfish proposes for the source and follow-up inputs is used to perform new evaluations, yielding e'_1 and e'_2. We compare these values in the definition of the MR.

In addition to the previous MRs, we defined other MRs that were not used in the experiments reported in this paper but also have the potential to find inconsistent behaviours. We briefly describe them.

- We considered $\texttt{MR}_{\texttt{equi}}$ but applying a variation of the `sim_diag` transformation. The idea is to rotate all the pieces with respect to the $a8 - h1$ diagonal (instead of with respect to $a1 - h8$ diagonal, as in the `sim_diag` transformation).
- We considered $\texttt{MR}_{\texttt{first}}$ but also applying the `sim_diag` and `sim_mirror` transformations.
- The inverse of $\texttt{MR}_{\texttt{better}}$. Given a source input p, we replace a queen by either a rook or a bishop of the same colour. The idea is that the evaluation obtained from p should be better than the one corresponding to the follow-up input.
- The composition of two transformations (e.g. the composition of $\texttt{MR}_{\texttt{first}}$ and $\texttt{MR}_{\texttt{mirror}}$ or of $\texttt{MR}_{\texttt{equi}}$ and $\texttt{MR}_{\texttt{mirror}}$).
- We can consider a variation of $\texttt{MR}_{\texttt{first}}$ where we compare the evaluations after performing the move suggested by the engine and after performing a different move.
- Since Stockfish returns the best suggested move m, the evaluation of p should be similar to the evaluation of p `AFTER` m.

4.2. Implementation details

We have implemented our MRs and applied them to different positions evaluated in Stockfish. Next, we briefly describe the main features of the programming language that we have used and present sample code. Our MRs have been implemented in Python, an object-oriented and high-level programming language with dynamic semantics. We chose this language because of our expertise (we have extensively used it in previous work), the existence of the previously mentioned *Stockfish* package in Python that strongly simplifies the interaction with the chess engine, and the enormous number of libraries that ease the development of functions that implement our MRs. Thanks to the *Stockfish* package, Python allows an input evaluation to be defined as a function. Consider the code given in Fig. 5(b). In the first line, we begin by defining the function. In lines 2 to 5, we initialise the ELO, depth, and position. In the sixth line, we specify the return of the function, which is the evaluation of the position and the best move to make. The seventh line is a call to this function, using a board, an ELO equal to 3700, a depth equal to 10, and the last two lines are the result of the evaluation. In this case, there is a White advantage equal to 31 centipawns and the first suggested move is ♘$c2 - c4$. The result, after performing this move, is that the White advantage is equal to 26 centipawns and no forced checkmate has been detected.

In order to apply our MRs, we need to build follow-up inputs from source inputs. This is done by using the transformation rules defined in the previous section. For example, the code to compute the `sim_axis` transformation used in the $\texttt{MR}_{\texttt{equi}}$ MR is given in Fig. 5(e). In the first line, we define the function. From lines 2 to 6, we define four auxiliary variables. In lines 7 and 8, we use a for loop to add the rows to an empty list in reverse order. In lines 9 and 10, we structure the returned position appropriately. Finally, in lines 11 to 14, we add

```python
def MR_equi(o1,o2,delta,epsilon):
    final = bool()
    if o1.get('type') != o2.get('type'):
        final = False
    elif o1.get('type') == o2.get('type') == 'mate' and o1.get('valu
        final = False
    elif o1.get('type') == o2.get('type') == 'cp' and dif1(o1.get('
                                delta and dif2(o1.get('value'),
        final = False
    else:
        final = True
    return final
```

(a)

```python
def sim_axis(pos):
    pos_transformed = []
    col = pos[-1:]
    aux1 = pos[:-2]
    aux2 = aux1.split("/")
    aux3 = []
    for j in range(8):
        aux3.append(aux2[j][::-1])
    aux4 = '/'.join(aux3)
    pos_transformed.append(aux4)
    if col == 'w':
        pos_transformed = pos_transformed[0] + ' w'
    if col == 'b':
        pos_transformed = pos_transformed[0] + ' b'
    return pos_transformed

sim_axis('6k1/2p1rrp1/1p1p4/8/8/1PPN1B1P/4PPP1/3NBK2 w')
```

```
'1k6/1prr1p2/4p1p1/8/8/P1B1NPP1/1PPP4/2KBN3 w'
```

(e)

```python
def evaluation(pos, elo, depth):
    stockfish = Stockfish('stockfish_15_x64_avx2.exe')
    stockfish.set_elo_rating(elo)
    stockfish.set_depth(depth)
    stockfish.set_fen_position(pos)
    return stockfish.get_evaluation(), stockfish.get_top_moves(1)
evaluation('rnbqkb1r/ppp1pp1p/5np1/3p4/3P4/5N2/PPP1PPPP/RNBQKB1R w', 3700, 10)
```

```
({'type': 'cp', 'value': 31},
 [{'Move': 'c2c4', 'Centipawn': 26, 'Mate': None}])
```

(b)

```python
pos = '6k1/2p1rrp1/1p1p4/8/8/1PPN1B1P/4PPP1/3NBK2 w'
MR_equi(evaluation(pos,4000,10)[0],evaluation(sim_axis(pos),4000,10)[0],0.1,10)
```

```
True
```

(c)

```python
pos = '6k1/2p1rrp1/1p1p4/8/8/1PPN1B1P/4PPP1/3NBK2 w'
MR_equi(evaluation(pos,4000,10)[0],evaluation(sim_axis(pos),4000,10)[0],0.01,5)
```

```
False
```

(d)

Fig. 5. Fragments of code.

the corresponding colour of the pieces that move to the position. The last two lines are a call to the function and the corresponding result. Both positions, assuming that White moves, are graphically depicted in Fig. 2.

Finally, in Fig. 5(a) we show the code corresponding to the implementation of $\texttt{MR}_{\texttt{equi}}$. In the first line, we define the function. In the second line, we create a Boolean variable named `final`, which serves as the function output. In lines 3 and 4, we assign the value false to `final`, based on the evaluation function returning different *types*. In lines 5 and 6, we also assign false to `final`, based on the evaluation function returning *mate* in both cases, but with different *values*. In lines 7 to 9, we again assign false to `final`, based on the evaluation function returning *cp* in both cases, but with *values* that do not meet the thresholds. In lines 10 and 11, we assign true to `final` in all other cases.

Observe that the implementation of our MRs only has four parameters: two outputs and two thresholds. Note that these functions are invoked from another function that traverses the dataset to obtain source inputs, perform valid transformations to create follow-up inputs, and invokes the previous function using as parameters the evaluation of the previous inputs. For example, if we consider the first two positions given in Fig. 2, p and $\texttt{sim_axis}(p)$, we have that the MR is not violated if we use $\delta = 0.1$ and $\epsilon = 10$. The code associated with the application of this MR can be found in Fig. 5(c). However, if we use tighter thresholds ($\delta = 0.01$ and $\epsilon = 5$) then the MR is violated. The code associated with the application of this MR can be found in Fig. 5(d).

All the code used to perform the experiments, in particular the implementation of our MRs and the datasets, is in https://github.com/MMH1997/MT_ChessEngines.

5. Research questions and experiments

In order to assess the usefulness of our approach, we performed a large battery of experiments on Stockfish. First, we pose the research questions that we try to answer with our experiments. Next, we describe how we collected and processed the source inputs and the results of the experiments and lessons learned. Finally, we provide answers to our research questions and evaluate the potential threats to the validity of the results.

5.1. Research questions

The primary goal of our work is to evaluate whether chess engines, in our case Stockfish 15, show inconsistent behaviours. Therefore, our first research question must be the following:

Research Question 1. *Is our MT approach an effective way to find errors in Stockfish 15?*

As we indicated in the introduction of the paper, we were aware of faulty behaviours in Stockfish 15 before we defined and implemented our tool but this does not imply that results deviating from the correct ones will be revealed by our tool. If the answer to this question is positive, then we have a follow-up question:

Research Question 2. *How critical are the problems detected by our tool?*

We considered four categories of deviations from the correct result, from more to less critical. We describe them for our first MR, $\texttt{MR}_{\texttt{equi}}$, but a similar classification can be given for the other MRs. We assume that equivalent positions are evaluated by using the same ELO and depth.

1. The evaluation of a position indicates that White/Black has an advantage of x centipawns, the evaluation of an equivalent position (via a `sim_axis` or `sim_diag` transformation) indicates that White/Black has an advantage of $-y$ centipawns, with $x, y \in \mathbb{N}$, and $x + y$ is *big*. Similarly, if White/Black has an advantage

of x centipawns, then the evaluation of a mirror position (via a `sim_mirror` transformation) indicates that White/Black has an advantage of y centipawns, with $x, y \in \mathbb{N}$, and $x + y$ is *big*.
2. The evaluation of two equivalent positions is "mate in x moves" in one of them while not obtaining a "mate in y moves" for the other one.
3. The evaluation of two equivalent positions is "mate in x moves" and "mate in y moves", with $x \neq y$. The bigger $|x - y|$ is, the more critical the problem.
4. The evaluation of two equivalent positions give the advantage to the same side but the (relative) difference between the evaluations is relevant.

The first category is the most critical one because our chess engine would evaluate two equivalent positions concluding that in one of them there is a clear White advantage while in the second one there is a clear Black advantage. We think that the second category is also critical because the chess engine would not be able to detect a checkmate, while such a checkmate does exist and it is found in an equivalent position.

The next question concerns scalability.

Research Question 3. *How well does our tool scale?*

The use of thresholds to define our MRs poses another question.

Research Question 4. *How sensitive are our MRs to the choice of thresholds?*

Finally, we would like to explore whether our solution can deal with other chess engines. Therefore, our final research question is:

Research Question 5. *Can our MT solution be applied to other chess engines?*

5.2. Data collection and pre-processing

In this section, we describe the data used in the experiments. In order to ensure heterogeneity of our experiments, we used positions obtained from three different open source repositories.

- The *Test Set Chess Positions* [44] repository. These positions were randomly generated and include boards from five to fifteen pieces: two kings (one of each colour), and from three to thirteen pieces (different to kings). This repository contains a total of 20,000 positions.
- The *Arasan Test Suite* (version 20) [45]. It contains a total of 200 positions from human and engine games. Most of the positions are non-trivial and some of them are *avoid move* positions, that is, positions where a bad move is very tempting.
- An *epd* file available at [46]. This repository has 418 positions from games with win result (for either side) in games between two powerful chess engines, Stockfish and Leela Chess Zero, in the 2019 superfinal of the TCEC.

The initial dataset has 20,618 positions. In addition, we doubled the number of positions by considering for each position the next move made by both players (White and Black). Therefore, we have as raw data a dataset with 41,136 positions. However, not all of these positions were useable and we needed a pre-processing phase. In particular, some of the randomly generated positions extracted from the first mentioned dataset were invalid. Therefore, we first evaluated all the positions and their valid transformations and if Stockfish did output an *exception* in either the original or the transformed position, then we discarded that pair. The next steps were more subtle and are related to checks. We discarded checkmate positions. Although it is valid to represent a checkmate position, it makes no sense to evaluate such positions because there are no possible valid next moves. In some cases, the first step of our pre-processing phase was able to discard some positions in which both kings are in check. These positions are invalid and, therefore, we discarded them.[3] However, we observed that Stockfish did not detect these invalid positions. Interestingly enough, we were not looking for this kind of errors and we just found them (before we applied MT). Therefore, we implemented a function to decide whether both kings are in check or not. The last step of our pre-processing phase was to apply this function and discard the pairs of positions where either the original or the transformed position showed that both kings were in check.

5.3. Results of the experiments

In this section we outline the results of our experiments. Our preliminary experiments showed that the number of detected problems at very low (1 or 2) depths is very small because Stockfish can evaluate all the possible combinations. Still, there are some (small) deviations from the correct results. As the depth used in the experiments increases (up to 6 or 7), the number of problems quickly grows. Because of this, we decided to perform all our experiments at depth 10, a medium-high depth, so that we can compare the results obtained for different MRs. We also chose for all the experiments an ELO equal to 50000: we are using the maximum ELO available in Stockfish. Therefore, detected problems cannot be attributed to using a low ELO Stockfish.

In order to display the results of our experiments, we use tables in which the y-axis displays different values for the threshold corresponding to $dist_{rel}$ and the x-axis displays different values for the threshold corresponding to $dist_{abs}$ (see Definition 3). For example, if we set $\delta = 0.5$ and $\epsilon = 40$, we can see in Fig. 8 that the percentage of applications of the $\mathtt{MR}_{\mathtt{mirror}}$ MR that failed is 13.3%. Note that if $\delta > 1$ then $dist_{rel}(x, y, d)$ always holds. Actually, $\delta = 0.5$ is a very loose threshold. Similarly, an absolute difference of 1 pawn might not seem as much if we are comparing big values. In this case, however, the first threshold allows the MR to pass the experiment, that is, we might not have $dist_{abs}(x, y, 100)$ but we certainly have $dist_{rel}(x, y, 0.5)$.

Fig. 6 displays the percentage of erroneous behaviours detected by $\mathtt{MR}_{\mathtt{equi}}$ if we apply the $\mathtt{sim_axis}(p)$ transformation to the source inputs of the dataset. This percentage runs from 36.1% to 10.7%. We can conclude that δ has much more influence in the final results than ϵ. Although this is not shown in the table, the percentage of detected problems related to mate, that is, either one position is evaluated to mate in x and the other one is not or both positions are evaluated as mate in x and mate in y with $x \neq y$, is 10%. This value is computed by applying the MR with $\delta > 1$ because this threshold allows us, as previously commented, to discard all the potential deviations from the correct result related to centipawns. We apply this *trick* in all the experiments reported in this section.

Fig. 7 displays the values corresponding to $\mathtt{MR}_{\mathtt{equi}}$ using the $\mathtt{sim_diag}(p)$ transformation. In general, this transformation is able to reveal more deviations between equivalent positions, running from 43.5% to 13.8%. Again, the influence of δ is higher than the influence of ϵ. In this case, the percentage of detected problems related to mate is 13.2%.

Fig. 8 displays the percentage of deviations in the evaluation of *symmetric* inputs using $\mathtt{MR}_{\mathtt{mirror}}$. This percentage runs from 37.6% to 12.8%. Again, the influence of ϵ is very small when compared to δ. Finally, if we set $\delta = 1$ then we obtain the percentage of detected problems related to mate: 12.1%.

Fig. 9 displays the values corresponding to $\mathtt{MR}_{\mathtt{better}}$. In this case, the number of detected problems is much smaller than in the rest of

[3] Technically speaking, and according to FIDE laws [43], epigraph 7.4.a, in such a situation we should move backwards to the previous (legal) position. The problem is that if we simply have the distribution of pieces in the board, then we do not know the steps taken. Therefore, we discard these invalid positions.

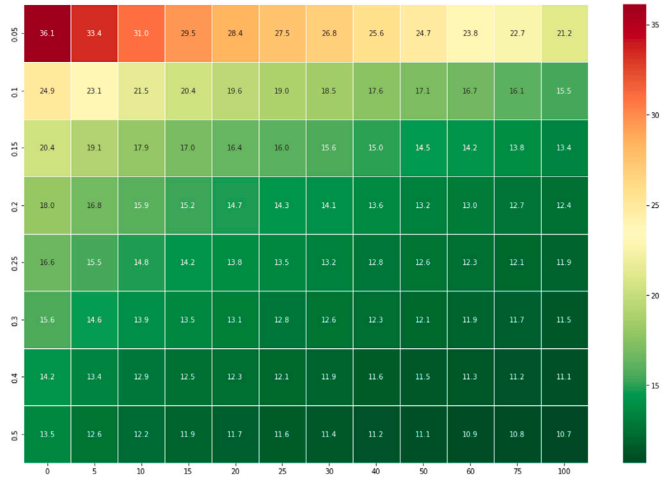


Fig. 6. Percentage of problems detected by $\texttt{MR}_{\texttt{equi}}$ using the $\texttt{sim_axis}(p)$ transformation. x-axis represents ϵ value and y-axis represents δ value.

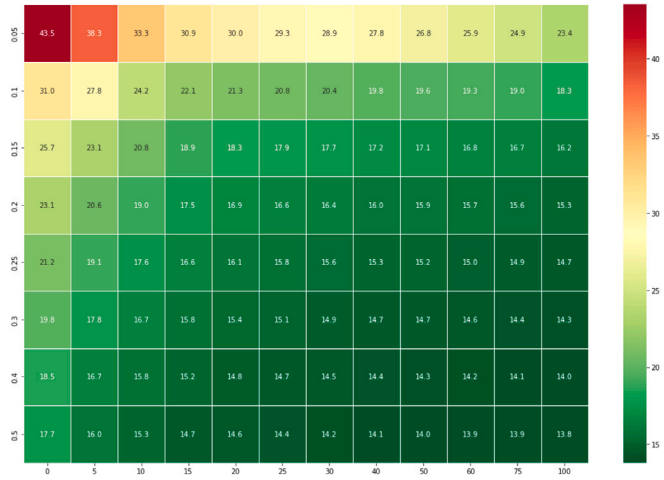


Fig. 7. Percentage of problems detected by $\texttt{MR}_{\texttt{equi}}$ using the $\texttt{sim_diag}(p)$ transformation. x-axis represents ϵ value and y-axis represents δ value.

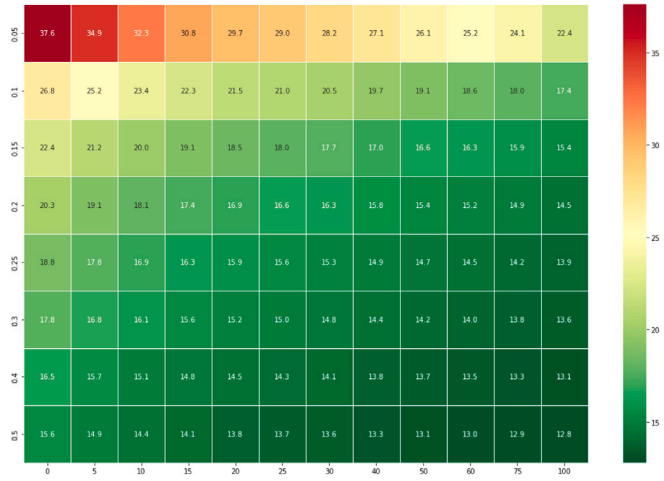


Fig. 8. Percentage of problems detected by $\texttt{MR}_{\texttt{mirror}}$. x-axis represents ϵ value and y-axis represents δ value.

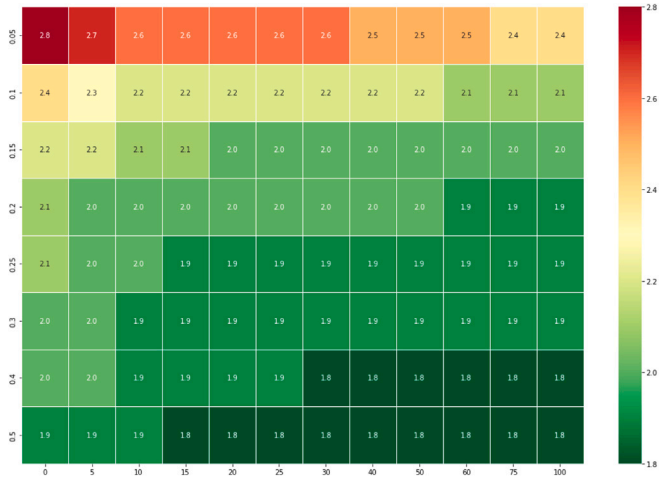


Fig. 9. Percentage of problems detected by $\texttt{MR}_{\texttt{better}}$. x-axis represents ϵ value and y-axis represents δ value.

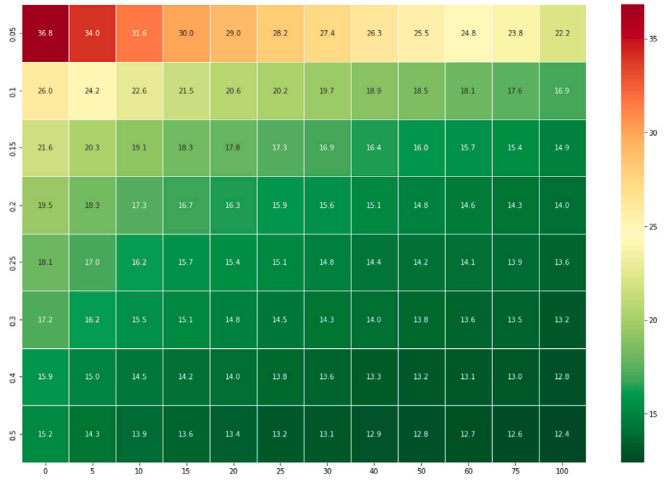


Fig. 10. Percentage of problems detected by $\texttt{MR}_{\texttt{first}}$ using the $\texttt{sim_axis}(p)$ transformation. x-axis represents ϵ value and y-axis represents δ value.

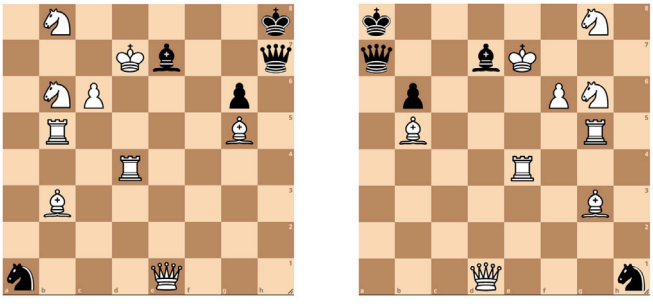
MRs: it runs from 2.8% to 1.8%. In this case, the influence of δ is small in the variation of the percentage of detected deviations while the influence of ϵ is virtually zero. In this case, although the percentage of problems related to mate is small, 1.7%, it still represents more than half of the detected problems. Note that, in order to ease the evaluation, we applied the `replace` transformation only to White.

Fig. 10 displays the values corresponding to $\texttt{MR}_{\texttt{first}}$. The results follow a pattern similar to the one corresponding to $\texttt{MR}_{\texttt{equi}}$ with the same transformation, but being the percentage of detected problems slightly higher in this case. This may happen because Stockfish has to evaluate the position, compute the best move and evaluate the position after this move, and this process is more complex than the ones when Stockfish only evaluates the given position. The percentage of relevant deviations between the evaluations corresponding to equivalent inputs runs from 36.8% to 12.4%. We have that the percentage of problems related to mate is equal to 11.9%.

After comparing the results of all MRs, it was found that $\texttt{MR}_{\texttt{better}}$ is the least effective MR, ranging from a detection rate of 2.8% with the tightest thresholds to 1.8% with the most conservative thresholds. This may be due to the fact that $\texttt{MR}_{\texttt{better}}$ focuses on a thorough understanding of the game itself. The remaining MRs have a similar level of effectiveness, with the percentage of detected problems ranging from around 40% to around 13%, depending on the threshold values.

Table 1
Enumeration of some non-negligible problems found by our MRs.

Figure	MR	Op. pos-A.	Op. pos-B	Eval. pos-A	Eval. pos-B	$\frac{\lvert x-y\rvert}{\lvert x\rvert+\lvert y\rvert}$	$\lvert x-y\rvert$	cat.
Fig. 11	MR_{equi}	'cp'	'cp'	7305	2850	0.435	4455	4
Fig. 12	MR_{equi}	'mate'	'mate'	−7	−5	–	–	3
Fig. 13	MR_{mirror}	'mate'	'cp'	9	−2101	–	–	2
Fig. 14	MR_{better}	'cp'	'cp'	207	−174	1	381	4
Fig. 15	MR_{first}	'cp'	'cp'	7318	2994	0.419	4324	4



(a) (b)

```
In [116]: pos = '1N5k/3Kb2q/1NP3p1/1R4B1/3R4/1B6/8/n3Q3 w'
          print(sim_axis(pos))

          k5N1/q2bK3/1p3PN1/1B4R1/4R3/6B1/8/3Q3n w

In [117]: evaluation(pos, 50000, 10)[0]
Out[117]: {'type': 'cp', 'value': 7305}

In [118]: evaluation(sim_axis(pos), 50000, 10)[0]
Out[118]: {'type': 'cp', 'value': 2850}
```

(c)

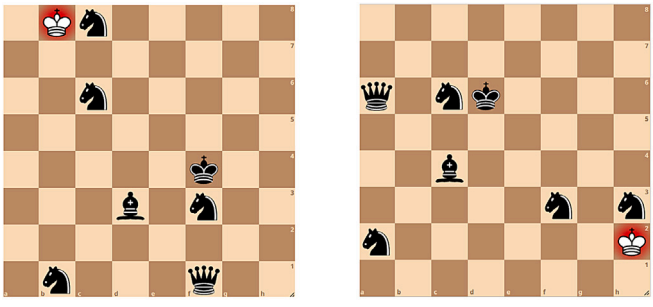
Fig. 11. Non-negligible problem found by MR_{equi}. Original position, p, (a); transformed position, $sim_axis(p)$ (b); and code used to obtain the evaluations (c).

Interestingly, MR_{equi} has the highest percentage of detected problems when using the $sim_diag(p)$ transformation, but also the lowest rate (excluding MR_{better}) when using the $sim_axis(p)$ transformation.

Table 1 presents the most severe deviations between evaluations found with our MRs. These are found even using very loose thresholds ($\delta = 0.4, \epsilon = 100$). Note that for the distances formula of MR_{mirror}, y must be multiplied by -1 while in MR_{better} we only consider distances such that $x > y$. This table displays three serious problems related to centipawns, another one related to difference in moves to mate and the last one related to an evaluation that outputs 'mate' while an equivalent one returns a number of 'centipawns'. In the first column we indicate the figure associated with the positions showing the detected problem and the code used to evaluate them (see Figs. 11–15).

We performed an additional experiment to evaluate the increase of the time needed to complete the evaluation of our MRs when we use different depths. We used the first 200 inputs of our dataset, applied the `sim_axis` transformation and evaluated MR_{equi}. Fig. 16 displays the time (in seconds) spent to perform the 200 evaluations by depth (from 1 to 20).

The goal of our work was to provide an MT solution that could evaluate chess engines. Our main focus was on the best available chess engine, Stockfish, but it would be desirable to check whether our solution is flexible enough to be applied to other chess engines. We found a Python package, *python-chess*, that can be used with other chess engines, not just Stockfish. The drawback is that in *python-chess* we cannot set parameters like ELO or depth, as they are different for each engine. Instead, the *time* parameter in the *python-chess* package serves



(a) (b)

```
In [180]: pos = '1Kn5/8/2n5/8/5k2/3b1n2/8/1n3q2 w'
          print(sim_diag('1Kn5/8/2n5/8/5k2/3b1n2/8/1n3q2 w'))

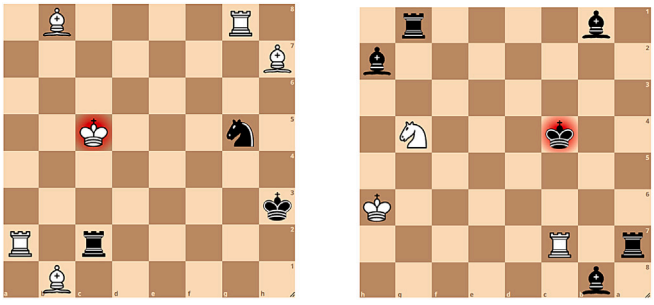
          8/8/q1nk4/8/2b5/5n1n/n6K/8 w

In [181]: evaluation(pos, 50000, 10)[1]
Out[181]: [{'Move': 'b8c8', 'Centipawn': None, 'Mate': -7}]

In [182]: evaluation(sim_diag(pos), 50000, 10)[1]
Out[182]: [{'Move': 'h2g3', 'Centipawn': None, 'Mate': -5}]
```

(c)

Fig. 12. Non-negligible problem found by MR_{equi}. Original position, p, (a); transformed position, $sim_diag(p)$ (b); and code used to obtain the evaluations (c).



(a) (b)

```
In [119]: pos = '1B4R1/7B/8/2K3n1/8/7k/R1r5/1B6 w'
          print(sim_mirror(pos))

          1b6/r1R5/7K/8/2k3N1/8/7b/1b4r1 b

In [120]: evaluation(pos, 50000, 10)[0]
Out[120]: {'type': 'mate', 'value': 9}

In [121]: evaluation(sim_mirror(pos), 50000, 10)[0]
Out[121]: {'type': 'cp', 'value': -2101}
```

(c)

Fig. 13. Non-negligible problem found by MR_{mirror}. Original position, p, (a); transformed position, $sim_mirror(p)$ (b); and code used to obtain the evaluations (c).

a similar purpose as *ELO* and *depth* in the *Stockfish* package. Thus, even though the formal definition of our MRs (see Section 4.1) does not need to be modified, certain parts of the code implementing them need to be adapted to use *python-chess*.

We need to modify how we obtain the evaluation result from the engine, as outlined in Section 4.2. When using the *Stockfish* package, the `get_evaluation` function returns a dictionary that displays the

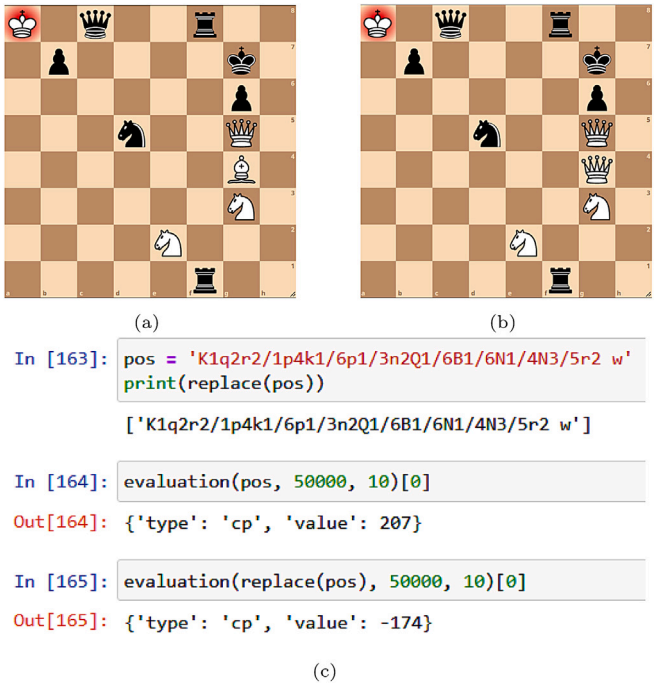


Fig. 14. Non-negligible problem found by $\mathrm{MR}_{\mathrm{better}}$. Original position, p, (a); transformed position, $\mathrm{REPLACE}(p)$ (b); and code used to obtain the evaluations (c).

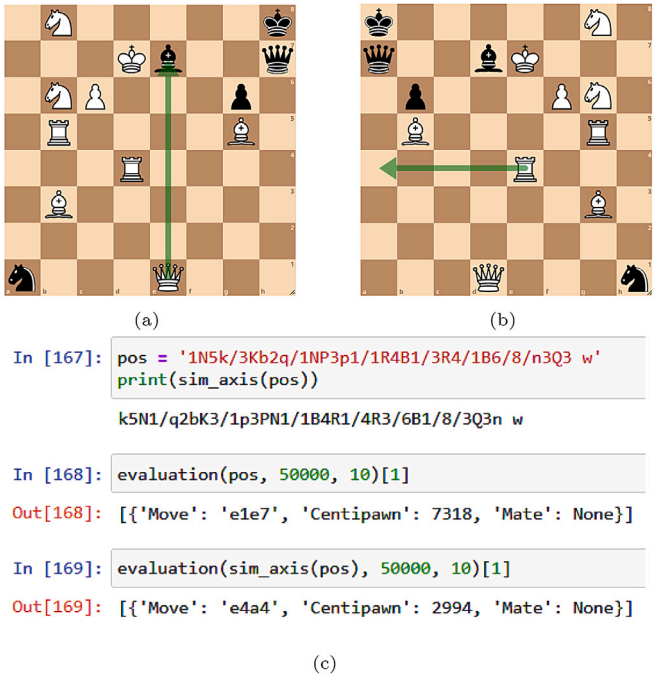


Fig. 15. Non-negligible problem found by $\mathrm{MR}_{\mathrm{first}}$. Original position, p, (a); transformed position, $\mathtt{sim_axis}(p)$ (b); and code used to obtain the evaluations (c).

type and *value* of the position evaluation, as well as a list of the best move recommended by the engine and its characteristics. However, when using the *python-chess* package, the `engine.analyse` function returns a data type called *PovScore*. We cannot work directly with this type of data as we did with the *Stockfish* package. Therefore, we must convert it to a string and use regular expressions to extract the *type* and *value* of the position.

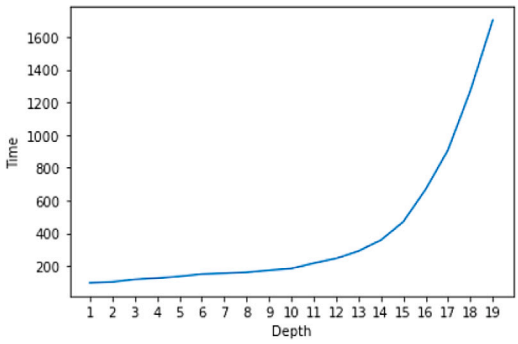


Fig. 16. Time (in seconds) by depth in the evaluation of 200 inputs (applying the $\mathrm{MR}_{\mathrm{equi}}$ MR with the $\mathtt{sim_axis}(p)$ transformation).

Table 2
% of problems detected after applying $\mathrm{MR}_{\mathrm{equi}}$ ($\mathtt{sim_axis}$), $\mathrm{MR}_{\mathrm{first}}$, $\mathrm{MR}_{\mathrm{mirror}}$ and $\mathrm{MR}_{\mathrm{better}}$ using $\delta = 0.25$ and $\epsilon = 25$ as thresholds.

Engine	$\mathrm{MR}_{\mathrm{equi}}$ ($\mathtt{sim_axis}$)	$\mathrm{MR}_{\mathrm{first}}$	$\mathrm{MR}_{\mathrm{mirror}}$	$\mathrm{MR}_{\mathrm{better}}$
Komodo-13	26.41%	34.60%	20.75%	23.52%
Stockfish 15	16.67%	40.28%	7.40%	3.70%
Houdini 15	25.00%	67.77%	11.32%	17.60%
Gull 3	18.87%	0.00%	22.64%	0.00%

The implementation of the $\mathrm{MR}_{\mathrm{first}}$ MR needed some additional changes. The `engine.analyse` function, included in the *python-chess* package, returns the recommended next move but not the evaluation after the next move. Thus, we created an additional `next_move` function that takes a board and a move as input and returns the board after the move. We apply this function to the best move recommended by the engine and the original board. Then, we evaluate the output obtained. Note that in our previous experiments, when we used the *Stockfish* package, we did not require the additional function because the `get_evaluation` function also returned the evaluation of the position after the move was recommended.

After the new setting was in place, we applied our MRs to four different chess engines using the *python-chess* package: Stockfish 15, Gull, Komodo, and Houdini. We applied $\mathrm{MR}_{\mathrm{equi}}$ (using $\mathtt{sim_axis}$), $\mathrm{MR}_{\mathrm{better}}$, $\mathrm{MR}_{\mathrm{mirror}}$ and $\mathrm{MR}_{\mathrm{first}}$ to the first 100 positions of our dataset. Since all these engines but Stockfish are non-deterministic, we calculated the average of 10 repetitions. Concerning the thresholds, we used a medium value: $\delta = 0.25$ and $\epsilon = 25$. Table 2 displays the percentage of detected problems found in each engine by MR.

Although the goal of this experiment was to show that, after minor changes, we can apply our tool to other chess engines, we can analyse the results. The number of problems detected by $\mathrm{MR}_{\mathrm{mirror}}$ and $\mathrm{MR}_{\mathrm{better}}$ is small and the percentages are similar. Next is $\mathrm{MR}_{\mathrm{equi}}$, with problem detection percentages around 25%. Finally, $\mathrm{MR}_{\mathrm{first}}$ is the MR finding more problems. These values are in line with what we obtained during our experiments in Stockfish using the dedicated *Stockfish* package.

In order to perform a specific analysis per chess engine, we have to set Gull apart. In contrast to the other three engines, Gull does not use *mate*: if checkmate can happen in a few moves, Gull shows a very high *cp* value. Thus, technically speaking, we can only consider a category of problems for Gull, category 4, although very big differences between *cp* evaluations should be considered as problems belonging to the first three categories. As Gull only detects one category of problems, we focus the analysis on the other three engines. Stockfish is the engine in which fewer problems are detected in general and in three out of four MRs. Komodo and Houdini have different behaviours: while Komodo has a more or less constant percentage of detected problems, regardless of the MR, Houdini has the highest percentage of problems detected by $\mathrm{MR}_{\mathrm{equi}}$, but it also has low percentages of problems detected in $\mathrm{MR}_{\mathrm{mirror}}$ and $\mathrm{MR}_{\mathrm{better}}$.

Table 3
Evaluations of p after move-A (original move in Carlsen-Nepo) and of sim_mirror(p) after move-B (move that should be done to achieve the transformed position). The sixth and seventh columns show the *distances* between the results. The last column shows whether MR_{mirror} is violated by using $\delta = 0.25$ and $\epsilon = 25$ as thresholds. Note that since we are evaluating p and sim_mirror(p), we need to multiply by -1 one of the centipawn values (that is the reason why we have $-y$ in the formulas).

Carlsen vs Nepo, WCC 2021, game 6							
id	move-A	move-B	ev. move-A	ev. move-B	$\frac{\lvert x-(-y)\rvert}{\lvert x\rvert+\lvert -y\rvert}$	$\lvert x-(-y)\rvert$	MR_{mirror} violated?
1	d2d4	d7d5	'type': 'cp', 'value': 42	'type': 'cp', 'value': −24	0.272	18	NO
1	g8f6	g1f3	'type': 'cp', 'value': 12	'type': 'cp', 'value': −4	0.5	8	NO
2	g1f3	g8f6	'type': 'cp', 'value': 35	'type': 'cp', 'value': −35	0	0	NO
2	d7d5	d2d4	'type': 'cp', 'value': 20	'type': 'cp', 'value': −9	0.379	11	NO
3	g2g3	g7g6	'type': 'cp', 'value': 30	'type': 'cp', 'value': −20	0.2	10	NO
3	e7e6	e2e3	'type': 'cp', 'value': −5	'type': 'cp', 'value': 10	0.333	5	NO
4	f1g2	f8g7	'type': 'cp', 'value': 51	'type': 'cp', 'value': 10	1	61	YES
4	f8e7	f1g2	'type': 'cp', 'value': 23	'type': 'cp', 'value': 47	1	70	YES
5	0–0	0–0	'type': 'cp', 'value': 50	'type': 'cp', 'value': −23	0.37	27	YES
5	0–0	0–0	'type': 'cp', 'value': 27	'type': 'cp', 'value': −121	0.635	94	YES
6	b2b3	b7b6	'type': 'cp', 'value': 39	'type': 'cp', 'value': −60	0.212	21	NO
6	c7c5	c2c4	'type': 'cp', 'value': 4	'type': 'cp', 'value': −11	0.467	7	NO
7	d4c5	d5c4	'type': 'cp', 'value': 39	'type': 'cp', 'value': −18	0.368	21	NO
7	e7c5	e2c4	'type': 'cp', 'value': 0	'type': 'cp', 'value': 0	0	0	NO
8	c2c4	c7c5	'type': 'cp', 'value': 4	'type': 'cp', 'value': −25	0.724	21	NO
8	d5c4	d4c5	'type': 'cp', 'value': −15	'type': 'cp', 'value': −6	1	21	NO
9	d1c2	d8c7	type': 'cp', 'value': 22	'type': 'cp', 'value': −12	0.294	10	NO
9	d8e7	d1e2	'type': 'cp', 'value': 0	'type': 'cp', 'value': 0	0	0	NO
10	b1d2	b8d7	'type': 'cp', 'value': 17	'type': 'cp', 'value': −16	0.03	1	NO
10	b8c6	b1c3	'type': 'cp', 'value': 0	'type': 'cp', 'value': 4	1	4	NO

5.4. Answer to the research questions

In view of the results of our experiments, we provide informed answers to our research questions.

The answer to Research Question 1 is that our MRs found a considerable amount of problems. Even being extremely loose with the thresholds and discarding the detected problems related to centipawns, we have a percentage higher than 10% in all the MRs except in MR_{better}. We can conclude that our tool is an effective way to find wrong evaluations in Stockfish 15. The experiments have been performed using an adequate depth although not excessively high because of the scalability problem that we will discuss below. Then, we decided to perform another experiment using a much higher depth: 30. We choose a depth equal to 30 because the average time that Stockfish spends evaluating a position with this depth is around 3 minutes. Note that this is the average time by move in classical chess tournaments: 120 minutes per player for the first 40 moves. In this experiment, we applied MR_{mirror} to the first 20 positions of the game six of the World Chess Championship[4] between Magnus Carlsen and Ian Nepomniachtchi (the longest game in the history of World Chess Championship). We used medium threshold values ($\delta = 0.25$ and $\epsilon = 25$). Table 3 displays that during the first 20 positions of the game, MR_{mirror} is violated in 4 of them. From this additional experiment we can conclude that our tool is able to find faults regardless of the considered depth.

Concerning the answer to Research Question 2, severity of the detected errors, we were not able to find any highly critical error but we found deviations from the evaluation of related positions for all the MRs even when considering very loose thresholds. We think that problems in category 4, a significant deviation between the evaluation in centipawns of two related positions, should not be allowed but deviations related to mate are more severe. Table 4 displays the percentage of problems belonging to categories 2 and 3 detected by MR and by transformation. Recall that thresholds play no role in these categories, that is, the percentage is invariant if we increase/decrease the thresholds. As expected, the number of problems belonging to category 2 is smaller than the ones belonging to category 3. In particular, virtually all problems related to mate detected by MR_{better} belong to category 3.

Table 4
% of detected problems that belong to categories 2 and 3 found by MR and transformation.

MR (transformation)	Percentage of cat. 2 problems	Percentage of cat. 3 problems
MR_{equi} (sim_axis)	3.7%	6.3%
MR_{equi} (sim_diag)	3.6%	9.6%
MR_{mirror} (sim_mirror)	3.7%	8.4%
MR_{better} (replace)	0.2%	1.6%
MR_{first} (sim_axis)	3.2%	8.7%

In order to answer Research Question 3, we consider the results of the experiment given in Fig. 16. While the growth is very moderate for low depths, the growth is clearly exponential, as expected, from depth 14. In particular, after $depth = 15$, time doubles when depth is increased by two. Therefore, it is very likely that after a certain depth, we might not be able to conclude our evaluations. However, we have to mention that this is not a drawback of our tool: the time needed to apply our MRs is almost negligible and the bottleneck is the time needed by Stockfish to evaluate positions.

The answer to Research Question 4, influence of the chosen thresholds, is that the results are somehow sensitive to the values of the threshold associated with $dist_{rel}$ (that is, δ) but much less to the threshold associated with $dist_{abs}$ (that is, ϵ). This happens because δ deals with all data and it aims to normalise the difference. We use this threshold because, in practise, the difference between an evaluation of 50 and one of 150 centipawns is more important than the difference between an evaluation of 1500 and one of 1600 centipawns. Besides, ϵ only deals with small differences in absolute value. Therefore, ϵ affects a smaller number of evaluations than δ. Taking all these into account, the average percentage of problems detected by the MRs is 38.5% when we set the smallest thresholds and 12.4% in the most conservative case. We must also mention the particular case of MR_{better} (see Fig. 9) where the results are sensitive to modifications of δ, but not of ϵ. In fact, it might make sense to remove the second threshold in this MR but we keep it for symmetry with respect to the other MRs. It is worth to mention that while thresholds obviously affect the results, their impact is not so big as it was in our previous work [35] where we used them to set the size of a *big/small* set of nodes in a map.

In view of the experiments reported in the previous section, we can affirmatively answer our Research Question 5: our MT tool can be

[4] The complete game is available at https://www.chessgames.com/perl/chessgame?gid=2127373.

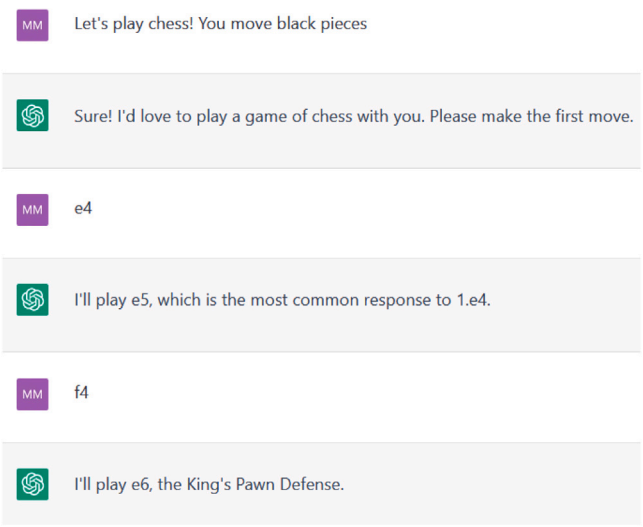


Fig. 17. The start of a chess game against ChatGPT.

applied to analyse other chess engines. This shows the versatility and flexibility of our solution because it was able to deal with other chess engines that were not initially considered. In particular, it is worth to mention that we were able to analyse a chess engine, Gull, that deviated from the behaviour of our original chess engine, Stockfish, because it does not use *mate*. We would like to emphasise that the changes to the original tool were minor: the formal definition of the MRs was not modified and only small changes, related to the parameters of the chess engines in the new package, were needed. Finally, considering the significant and recent impact of ChatGPT, a large language model developed by OpenAI, we decided to apply our MRs to ChatGPT. However, after conducting some preliminary experiments, we abandoned this idea because ChatGPT was unable to even correctly execute basic game-play movements. For example, Fig. 17 shows one interaction with ChatGPT where it initially moves $\triangle e7-e5$ and in the next turn it moves $\triangle e7-e6$, which is not allowed by the chess rules.

5.5. Threats to validity

In order to discuss the threats to the validity of the results of our experiments we divide them between threats to internal validity, to external validity, and to construct validity.

The threats to internal validity focus on uncontrolled factors that could have influenced the obtained results. In this category, our main concern is the possible faults present in the developed code. To reduce the impact of this threat, we used carefully constructed examples for which we could check the results, to thoroughly test the code implementing our MRs. Due to the high level of automation that our solution provides, we needed to ensure that the (automatic) generation of follow-up inputs was sound too. We, therefore, separately tested the code implementing our transformations. Finally, the last potential issue in this category concerns the defined MRs. It is obvious that a different set of MRs would obtain a different set of results, as the capability of our tool to detect problems in chess engines depends on the MRs used. To that end, we discussed our MRs with colleagues also working on MT and knowledgeable in chess as well as with chess experts, such as members of the current Spanish absolute team coach, who validated the equivalence of the MRs over the transformations we perform.

The threats to external validity focus on the generalisation capabilities of our results and their capability of being extrapolated to other situations. We have two concerns in this category: whether our MRs would be able to find problems in different situations, and whether our tool would work when applied in other situations (e.g. different notations to represent positions, other chess engines). To address the first concern, we explored our MRs over multiple chess positions, trying to be as diverse as possible. In this line, looking for heterogeneity, we considered both randomly generated positions and positions extracted from *real* games. To address the second concern, we considered the widely used FEN chess notation. In order to use a different notation, we would have to implement minor changes in our tool (mainly, modify the code corresponding to the allowed transformations so that the new notation is used). Concerning the application of our tool to other chess engines, as long as they provide evaluation of positions similar to Stockfish, the changes would be minor, as we have shown in our experiments. Regarding this, we observe that other chess engines are not deterministic. In this case, *typical* thresholds might need to be adjusted so that bigger deviations are allowed.

The threats to construct validity focus on the relevance of the properties that we are using in our MRs. In order to evaluate it, we have to measure the fault detection ability of the generated input values and the time needed to obtain and apply them. We diminish this threat by exploring multiple different chess positions with different number of pieces (and thus complexity). Having faults in our MRs definition or in our follow-up input generation algorithm are also threats to construct validity. To address them we executed a wide range of experiments that covered the analysed areas. Finally, a last construction threat is our dependency on an external tool to communicate with Stockfish and the other chess engines, what might complicate the applicability of our tool to other chess engines. We are aware of this threat and in the future we would like to adapt our code to work with any chess engine.

6. Conclusions and future work

Metamorphic Testing (MT) has been widely used to find faulty behaviours in systems for which we do not have an oracle. MT techniques compare the outputs produced for inputs that are somehow related. In this paper we have applied MT to find faults in chess engines, a clear example of complex programs without an oracle. This problem is interesting because no research work has previously focused on the correctness of such programs, although some work has studied their performance. Our solution aims to validate chess engines through the use of MRs that could find faults in their analysis. In fact, our experiments found that for the best chess engine, Stockfish, 10.3% of MR applications do not pass even when very loose thresholds are used. This has been a surprising result, as we were not expecting to find faults that generate such huge differences in analysis. Moreover, if we reduce the threshold, 31.36% of MR applications do not pass. Although we were expecting to find some faults that lead to small differences in the analysis, we did not think that we would uncover such a huge amount of them in these high-quality systems. These results clearly show that chess engines still have room for improvement, even if they consistently beat humans playing chess. They also show how our tool has been able to detect such faults and are useful to validate other chess engines.

Although we do not have concrete evidence, we can still speculate on the potential main cause for the detected failures. Chess engines explore moves in a specific order during the search process. However, if this order is different, for example, between a position and its symmetrical position, or if there is any randomness introduced within this order at any point, it could lead to differences in the evaluation due to variations in the lines of play considered.

As future work we would like to extend our MRs, with new proposals and alternatives, to improve the fault detection capabilities of our MT solution. We would also like to thoroughly explore which thresholds could be considered acceptable for the different MRs, in order to fix them and improve the usability of our tool by other users. Although we have already evaluated four different chess engines, it would be great to have access to the chess engine that was able to

clearly beat Stockfish: AlphaZero. Finally, we would like to explore MRs for other games, such as Go and shogi, to find faults also in engines made for such games. We expect that MR_{equi}, MR_{mirror} and MR_{first} should be easy to adapt to Go and shogi because they are based on board symmetries. However, we are aware that MR_{better} cannot be adapted for the Go game since in Go there is only one kind of piece. The adaptation of MR_{better} to shogi also requires some extra effort, as there are multiple pieces that can be exchanged. In addition, there are different relationships in Go and shogi, which can be exploited to generate new MRs, that could not be applied to chess. For example, there should be a specific MR encoding the importance of the grouping of pieces in Go. Another interesting extension of our work that we would like to undertake is to contact the developers of the different chess engines, especially Stockfish. If we receive positive feedback, we might be able to help them in fixing the errors and improving the implementation of their algorithms. An area of further investigation will involve exploring the potential adaptation of our approach to test systems engaged in game-like scenarios to other systems that can be conceptualised as a game between two players, namely an attacker and a defender [47,48].

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We would like to thank Marco T. Morazán and the anonymous reviewers of the previous version of this paper for the careful reading, interesting comments and suggestions that have helped us to further strengthen the paper.

References

[1] G.J. Myers, C. Sandler, T. Badgett, The Art of Software Testing, third ed., John Wiley & Sons, 2011.
[2] P. Ammann, J. Offutt, Introduction to Software Testing, second ed., Cambridge University Press, 2017.
[3] R.M. Hierons, K. Bogdanov, J.P. Bowen, R. Cleaveland, J. Derrick, J. Dick, M. Gheorghe, M. Harman, K. Kapoor, P. Krause, G. Luettgen, A.J.H. Simons, S. Vilkomir, M.R. Woodward, H. Zedan, Using formal specifications to support testing, ACM Comput. Surv. 41 (2) (2009) 9:1–9:76.
[4] M. Utting, A. Pretschner, B. Legeard, A taxonomy of model-based testing approaches, Softw. Test. Verif. Reliab. 22 (5) (2012) 297–312.
[5] A.R. Cavalli, T. Higashino, M. Núñez, A survey on formal active and passive testing with applications to the cloud, Ann. Telecommun. 70 (3–4) (2015) 85–93.
[6] E.T. Barr, M. Harman, P. McMinn, M. Shahbaz, S. Yoo, The oracle problem in software testing: A survey, IEEE Trans. Softw. Eng. 41 (5) (2015) 507–525.
[7] T.Y. Chen, T.H. Tse, New visions on metamorphic testing after a quarter of a century of inception, in: 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE'21, ACM, 2021, pp. 1487–1490.
[8] T.Y. Chen, S.C. Cheung, S.M. Yiu, Metamorphic testing: a new approach for generating next test cases, Tech. Rep. HKUST-CS98-01, Department of Computer Science, Hong Kong University of Science and Technology, 1998.
[9] T.Y. Chen, T.H. Tse, Z.Q. Zhou, Fault-based testing without the need of oracles, Inform. Softw. Technol. 45 (1) (2003) 1–9.
[10] S. Segura, D. Towey, Z.Q. Zhou, T.Y. Chen, Metamorphic testing: Testing the untestable, IEEE Softw. 37 (3) (2020) 46–53.
[11] S. Segura, G. Fraser, A.B. Sánchez, A. Ruiz-Cortés, A survey on metamorphic testing, IEEE Trans. Softw. Eng. 42 (9) (2016) 805–824.
[12] T.Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T.H. Tse, Z.Q. Zhou, Metamorphic testing: A review of challenges and opportunities, ACM Comput. Surv. 51 (1) (2018) 4:1–4:27.
[13] Top Chess Engine Championship, 2022, https://tcec-chess.com/, (Accessed 30 October 2022).
[14] Stockfish 15 - Strong open source chess engine, 2022, https://stockfishchess.org/, (Accessed 30 October 2022).
[15] A. Wald, Sequential tests of statistical hypotheses, Ann. Math. Stat. 16 (2) (1945) 117–186.
[16] A. Wald, J. Wolfowitz, Optimum character of the sequential probability ratio test, Ann. Math. Stat. 19 (3) (1948) 326–339.
[17] M. Vanthoor, SPRT testing, in: Creating the Rustic Chess Engine, 2022, https://rustic-chess.org/progress/sprt_testing.html, (Accessed 30 October 2022).
[18] A.E. Elo, The proposed USCF rating system, its development, theory, and applications, Chess Life XXII (8) (1967) 242–247.
[19] Z.Q. Zhou, T.H. Tse, M. Witheridge, Metamorphic robustness testing: Exposing hidden defects in citation statistics and journal impact factors, IEEE Trans. Softw. Eng. 47 (6) (2021) 1164–1183.
[20] N. Mouha, M.S. Raunak, D.R. Kuhn, R. Kacker, Finding bugs in cryptographic hash function implementations, IEEE Trans. Reliab. 67 (3) (2018) 870–884.
[21] S. Tolksdorf, D. Lehmann, M. Pradel, Interactive metamorphic testing of debuggers, in: 28th ACM SIGSOFT Int. Symposium on Software Testing and Analysis, ACM, 2019, pp. 273–283.
[22] Z. Zhang, P. Wang, H. Guo, Z. Wang, Y. Zhou, Z. Huang, DeepBackground: Metamorphic testing for deep-learning-driven image recognition systems accompanied by background-relevance, Inf. Softw. Technol. 140 (2021) 106701.
[23] M. Jia, X. Wang, Y. Xu, Z. Cui, R. Xie, Testing machine learning classifiers based on compositional metamorphic relations, Int. J. Performability Eng. 16 (1) (2020) 67–77.
[24] D. Pesu, Z.Q. Zhou, J. Zhen, D. Towey, A Monte Carlo method for Metamorphic Testing of machine translation services, in: IEEE/ACM 3rd Int. Workshop on Metamorphic Testing, MET'18, IEEE, 2018, pp. 38–45.
[25] J.M. Almendros-Jiménez, A. Becerra-Terón, M.G. Merayo, M. Núñez, Metamorphic testing of OpenStreetMap, Inf. Softw. Technol. 138 (2021) 106631.
[26] Y. Xu, Z.Q. Zhou, X. Zhang, J. Wang, M. Jiang, Metamorphic testing of named entity recognition systems: A case study, IET Softw. 16 (4) (2022) 386–404.
[27] J.D. Ellis, R. Iqbal, K. Yoshimatsu, Verification of the neural network training process for spectrum-based chemical substructure prediction using metamorphic testing, J. Comput. Sci. 55 (2021) 101456.
[28] R. Abreu, J.P. Fernandes, L. Llana, G. Tavares, Metamorphic testing of oracle quantum programs, in: 3rd Int. Workshop on Quantum Software Engineering, Q-SE'22, IEEE, 2022, pp. 16–23.
[29] M. Jiang, T.Y. Chen, S. Wang, On the effectiveness of testing sentiment analysis systems with metamorphic testing, Inform. Softw. Technol. 150 (2022) 106966.
[30] Z. Hui, S. Huang, C. Chua, T.Y. Chen, Semiautomated metamorphic testing approach for geographic information systems: An empirical study, IEEE Trans. Reliab. 69 (2) (2020) 657–673.
[31] C. Murphy, K. Shen, G. Kaiser, Automatic system testing of programs without test oracles, in: 8th Int. Symposium on Software Testing and Analysis, ISSTA'09, ACM, 2009, pp. 189–200.
[32] X. He, X. Wang, J. Shi, Y. Liu, Testing high-performance numerical simulation programs: Experience, lessons learned, and open issues, in: 29th ACM SIGSOFT Int. Symposium on Software Testing and Analysis, ISSTA'20, ACM, 2020, pp. 502–515.
[33] S. Segura, J. Troya, A. Durán, A. Ruiz-Cortés, Performance metamorphic testing: Motivation and challenges, in: IEEE/ACM 39th Int. Conf. on Software Engineering: New Ideas and Emerging Technologies Results Track, ICSE-NIER'17, IEEE, 2017, pp. 7–10.
[34] A. Núñez, P.C. Cañizares, M. Núñez, R.M. Hierons, TEA-Cloud: A formal framework for testing cloud computing systems, IEEE Trans. Reliab. 70 (1) (2021) 261–284.
[35] J.M. Almendros-Jiménez, A. Becerra-Terón, M.G. Merayo, M. Núñez, Using metamorphic testing to improve the quality of tags in OpenStreetMap, IEEE Trans. Softw. Eng. 49 (2) (2023) 549–563.
[36] Z.Q. Zhou, L. Sun, T.Y. Chen, D. Towey, Metamorphic relations for enhancing system understanding and use, IEEE Trans. Softw. Eng. 46 (10) (2020) 1120–1154.
[37] A. Liaqat, M.A. Sindhu, A metamorphic relation based approach for testing a chess game, in: 14th Int. Conf. on Emerging Technologies, ICET'18, IEEE, 2018, pp. 1–6.
[38] A. Liaqat, M.A. Sindhu, G.F. Siddiqui, Metamorphic testing of an artificially intelligent chess game, IEEE Access 8 (2020) 174179–174190.
[39] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y.L. Traon, M. Harman, Mutation testing advances: An analysis and survey, in: A.M. Memon (Ed.), in: Advances in Computers, vol. 112, Elsevier, 2019, pp. 275–378.
[40] Tearth, A few thoughts about testing chess engines, 2022, https://tearth.dev/posts/a-few-thoughts-about-testing-chess-engines, (Accessed 30 October 2022).
[41] G. Van Rossum, F.L. Drake, Python 3 Reference Manual, CreateSpace, 2009.
[42] Forsyth-Edwards Notation, 2022, https://www.chessprogramming.org/Forsyth-Edwards_Notation, (Accessed 30 October 2022).
[43] FIDE laws of chess adopted at the 79th FIDE congress, 2009, https://www.fide.com/FIDE/handbook/LawsOfChess.pdf, (Accessed 30 October 2022).

[44] Kaggle chess positions, 2022, https://www.kaggle.com/datasets/koryakinp/chess-positions, (Accessed 30 October 2022).
[45] Arasan Test Suite, 2022, https://www.arasanchess.org/testsuite.shtml, (Accessed 30 October 2022).
[46] LC0 vs SF, 2022, https://github.com/fsmosca/chess-tests, (Accessed 30 October 2022).
[47] J. Roldán-Gómez, J. Boubeta-Puig, J. Carrillo-Mondéjar, J.M. Castelo Gómez, J. Martínez del Rincón, An automatic complex event processing rules generation system for the recognition of real-time IoT attack patterns, Eng. Appl. Artif. Intell. 123 (2023) 106344.
[48] J. Roldán-Gómez, J. Martínez del Rincón, J. Boubeta-Puig, J.L. Martínez, An automatic unsupervised complex event processing rules generation architecture for real-time IoT attacks detection, Wirel. Netw. (2023) in Press.